

The logo for CODERHOUSE, featuring the word "CODERHOUSE" in a bold, italicized, black, sans-serif font. The text is centered within a light gray rectangular background.

CODERHOUSE

Nombre del Proyecto: Proyecto Final

Nombre y Apellido: Lucila Nemesis Torres

Comisión: 81840

Introduccion:

La gestión eficiente de información es fundamental para cualquier negocio moderno. El presente proyecto consiste en el desarrollo de una base de datos relacional para una librería online, permitiendo registrar y organizar datos sobre libros, autores, clientes, ventas y sus detalles, con el fin de optimizar la operatoria y mejorar la toma de decisiones comerciales.

Objetivo:

El objetivo principal de este proyecto es diseñar y desarrollar un sistema de base de datos que permita centralizar y gestionar de manera precisa y eficiente la información de una librería. Esto incluye la administración de ventas, el seguimiento de stock, la consulta de historial de compras, y el soporte a informes y análisis de datos relevantes.

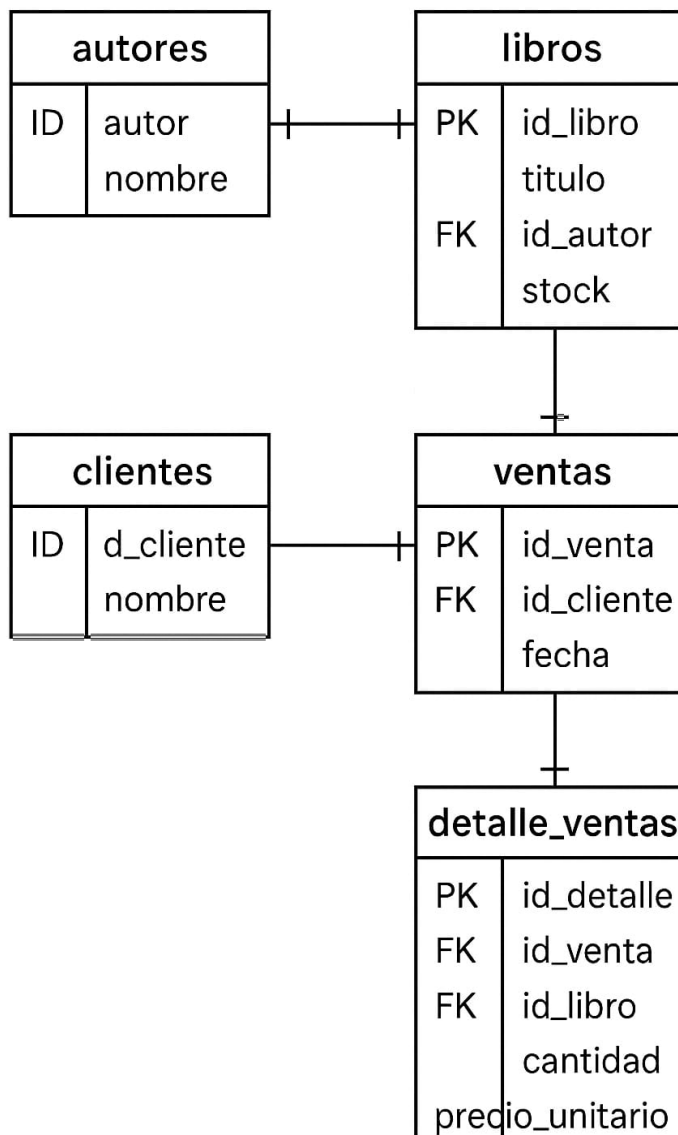
Situación problemática:

Actualmente, muchas librerías gestionan la información de forma manual o utilizando sistemas poco integrados, lo que genera errores, duplicidad de datos y dificultades para el análisis. La falta de una base de datos centralizada impide un correcto control de inventario, la rápida recuperación de información y la generación de reportes fiables para la toma de decisiones.

Modelo de negocio:

La librería comercializa libros de diferentes autores a clientes diversos a través de su tienda online y física. El sistema debe registrar cada venta, mantener actualizado el stock, y permitir acceder de manera ágil a información sobre autores, libros y compradores, así como generar reportes sobre el comportamiento de ventas y preferencias de los clientes.

Diagrama Entidad-Relación (E-R)



Breve explicación: El diagrama E-R muestra las entidades principales (autores, libros, clientes, ventas, detalle de ventas) y las relaciones entre ellas, representando la estructura de la base de datos sobre la cual se implementa el sistema.

Listado de tablas y estructura

Tabla: autores

- id_autor: Identificador único de autor (INT, PK, AUTO_INCREMENT)

- nombre: Nombre del autor (VARCHAR(80))

Tabla: libros

- id_libro: Identificador único de libro (INT, PK, AUTO_INCREMENT)
- titulo: Título del libro (VARCHAR(100))
- id_autor: Autor del libro (INT, FK)
- stock: Stock disponible del libro (INT)

Tabla: clientes

- id_cliente: Identificador único de cliente (INT, PK, AUTO_INCREMENT)
- nombre: Nombre completo del cliente (VARCHAR(100))
- Tabla: ventas
- id_venta: Identificador único de venta (INT, PK, AUTO_INCREMENT)
- id_cliente: Cliente que realiza la compra (INT, FK)
- fecha: Fecha de la venta (DATE)

Tabla: detalle_ventas

- id_detalle: Identificador del detalle de venta (INT, PK, AUTO_INCREMENT)
- id_venta: Venta asociada (INT, FK)
- id_libro: Libro vendido (INT, FK)
- cantidad: Cantidad de ejemplares vendidos (SMALLINT UNSIGNED)
- precio_unitario: Precio del libro en la venta (DECIMAL(8,2))

Scripts de creación de objetos

(Aquí luego pegaremos los bloques de código de creación de tablas, vistas, funciones, procedimientos y triggers, o bien poner “Ver archivo adjunto .sql” y solo una breve explicación)

Scripts de inserción de datos

A continuación se presentan los comandos SQL utilizados para cargar datos de ejemplo en cada una de las tablas principales del modelo:

-- Insertar autores

INSERT INTO autores (nombre) VALUES

('J.K. Rowling'),

('George Orwell');

-- Insertar libros

```
INSERT INTO libros (titulo, id_autor, stock) VALUES  
(  
'Harry Potter', 1, 10),  
(  
'1984', 2, 12);
```

-- Insertar clientes

```
INSERT INTO clientes (nombre) VALUES  
(  
'Juan Perez'),  
(  
'Ana Gomez');
```

-- Insertar ventas

```
INSERT INTO ventas (id_cliente, fecha) VALUES  
(  
1, '2025-12-15'),  
(  
2, '2025-12-16');
```

-- Insertar detalle de ventas

```
INSERT INTO detalle_ventas (id_venta, id_libro, cantidad, precio_unitario)  
VALUES  
(1, 1, 2, 500.00),  
(3, 2, 1, 600.00),  
(5, 3, 2, 550.00),  
(7, 4, 1, 700.00),  
(9, 5, 2, 800.00),  
(10, 6, 3, 450.00),  
(12, 7, 1, 670.00),  
(2, 8, 2, 520.00),  
(4, 9, 1, 710.00),  
(6, 10, 2, 640.00),  
(8, 11, 1, 500.00),  
(11, 12, 2, 900.00),  
(13, 1, 1, 600.00);
```

10. *Listado y descripción de vistas, funciones, procedimientos y triggers*

(Esta es la sección que ya te armé antes, podés pegar ahí cada descripción de los objetos creados y explicar su utilidad)

Ejemplo de informe generado

En este apartado se incluye un ejemplo de análisis basado en la estructura y los objetivos del sistema, utilizando la información que la base de datos permitirá obtener una vez que esté poblada con datos reales.

Objetivo del informe analítico

Obtener métricas que permitan identificar las ventas más relevantes, los libros más vendidos y los clientes que más compran, a partir de la información centralizada en la base de datos.

Ejemplo de consulta analítica:

Consulta SQL utilizada:

```
SELECT l.titulo, SUM(dv.cantidad) AS total_vendido
FROM detalle_ventas dv
JOIN libros l ON dv.id_libro = l.id_libro
GROUP BY l.titulo
ORDER BY total_vendido DESC;
```

Resultado esperado:

Debido a que los datos de prueba están limitados o no hay registros completamente relacionados en este entorno, la consulta podría no arrojar filas en la tabla resultante. Sin embargo, si la base de datos está poblada adecuadamente, el informe mostrará qué libros son los más vendidos en la librería.

Interpretación y utilidad:

- Este informe permite a la gestión del negocio saber cuáles títulos tienen mayor demanda y, por ende, planificar mejor la reposición de stock y las estrategias comerciales.
- A futuro, se puede exportar la tabla de resultados a Excel para realizar gráficos o análisis visuales más detallados (por ejemplo, gráficos de barras comparando ventas de diferentes títulos).

Conclusión del análisis:

A través de consultas como la presentada, la base de datos permite obtener información valiosa para la toma de decisiones, analizando tendencias, comportamientos de clientes y eficiencia del negocio.

Herramientas y tecnologías utilizadas

MySQL Workbench

Excel para análisis

Draw.io para el diagrama E-R

Conclusiones

El desarrollo de la base de datos permitió optimizar la gestión de la librería, logrando mayor control, seguridad y capacidad de análisis de sus operaciones. El sistema es escalable y puede adaptarse a futuros requerimientos.